NOTE There are several instructions you can't `ONBUILD`: `FROM`, `MAINTAINER`, and `ONBUILD` itself. This is done to prevent Inception-like recursion in `Dockerfile` builds.

Pushing images to the Docker Hub

Once we've got an image, we can upload it to the Docker Hub. This allows us to make it available for others to use. For example, we could share it with others in our organization or make it publicly available.

NOTE The Docker Hub also has the option of private repositories. These are a paid-for feature that allows you to store an image in a private repository that is only available to you or anyone with whom you share it. This allows you to have private images containing proprietary information or code you might not want to share publicly.

We push images to the Docker Hub using the `docker push` command.

Let's build an image without a user prefix and try and push it now.

Listing 1.88: Trying to push a root image

```
$ cd static_web
$ sudo docker build --no-cache -t="static_web" .
. . .
Successfully built a312a2ed58c7
$ sudo docker push static_web
The push refers to a repository [docker.io/library/static_web]
c0121fc36460: Preparing
8591faa9900d: Preparing
9a39129ae0ac: Preparing
98305c1a8f5e: Preparing
0185b3091e8e: Preparing
ea9f151abb7e: Waiting
unauthorized: authentication required
```

What's gone wrong here? We've tried to push our image to the repository `static_web`, but Docker knows this is a root repository. Root repositories are managed only by the Docker, Inc., team and will reject our attempt to write to them as unauthorized. Let's try again, rebuilding our image with a user prefix and then pushing it.

Listing 1.89: Pushing a Docker image

```
$ sudo docker build --no-cache -t="jamtur01/static_web" .
$ sudo docker push jamtur01/static_web
The push refers to a repository [jamtur01/static_web] (len: 1)
Processing checksums
Sending image list
Pushing repository jamtur01/static_web to registry-1.docker.io
(1 tags)
. . .
```

This time, our push has worked, and we've written to a user repository, `jamtur01`

`/static_web`. We would write to your own user ID, which we created earlier, and to an appropriately named image (e.g., `youruser/yourimage`).

We can now see our uploaded image on the Docker Hub.

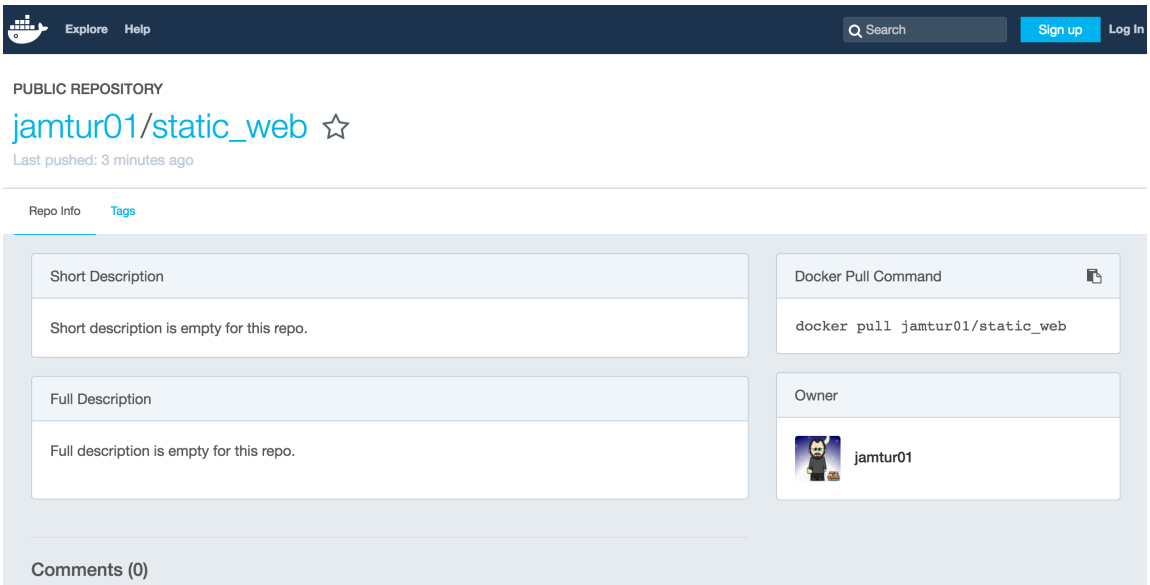

Figure 1.4: Your image on the Docker Hub.

TIP You can find documentation and more information on the features of the Docker Hub here.

Automated Builds

In addition to being able to build and push our images from the command line, the Docker Hub also allows us to define Automated Builds. We can do so by connecting a GitHub or BitBucket repository containing a `Dockerfile` to the Docker

Hub. When we push to this repository, an image build will be triggered and a new image created. This was previously also known as a Trusted Build.

NOTE Automated Builds also work for private GitHub and BitBucket repositories.

The first step in adding an Automated Build to the Docker Hub is to connect your GitHub account or BitBucket to your Docker Hub account. To do this, navigate to Docker Hub, sign in, click on your profile link, then click the `Create -> Create Automated Build` button.

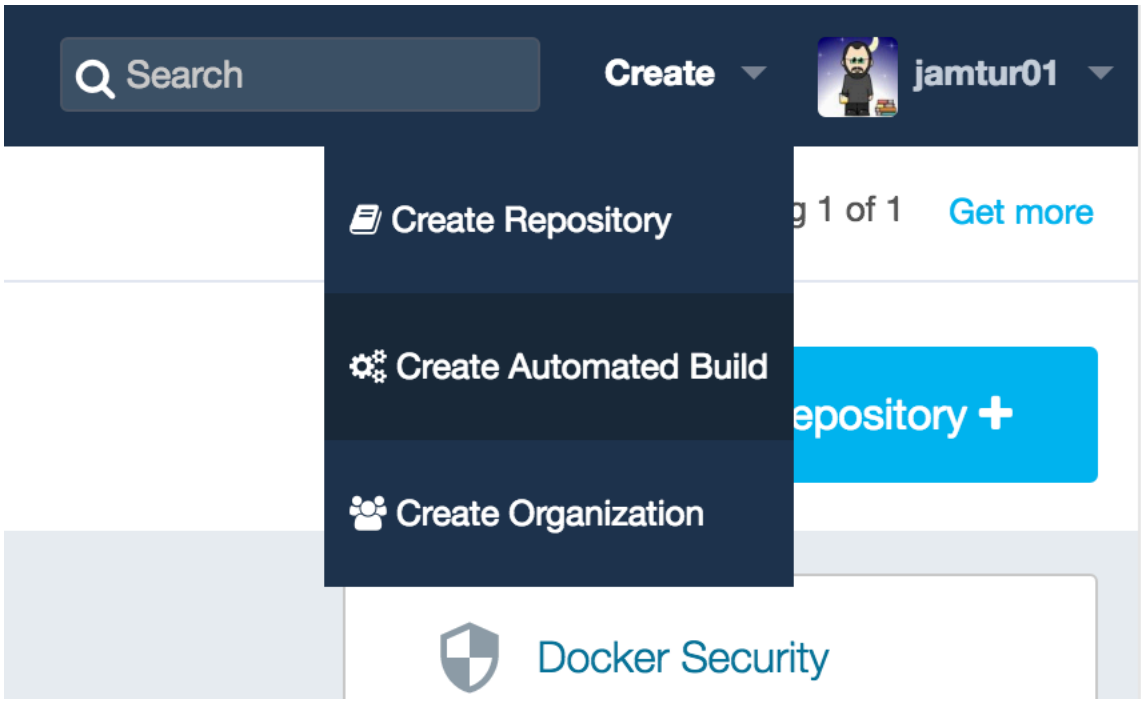

Figure 1.5: The Add Repository button.

You will see a page that shows your options for linking to either GitHub or Bit-

Bucket. Click the `Select` button under the GitHub logo to initiate the account linkage. You will be taken to GitHub and asked to authorize access for Docker Hub.

On Github you have two options: `Public and Private (recommended)` and `Limited`. Select `Public and Private (recommended)`, and click `Allow Access` to complete the authorization. You may be prompted to input your GitHub password to confirm the access.

From here, you will be prompted to select the organization and repository from which you want to construct an Automated Build.

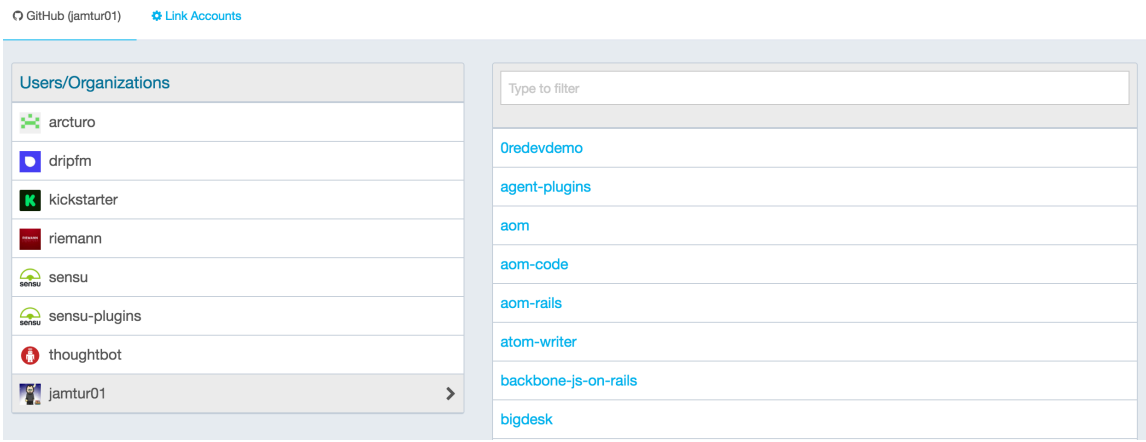

Figure 1.6: Selecting your repository.

Select the repository from which you wish to create an Automated Build and then configure the build.

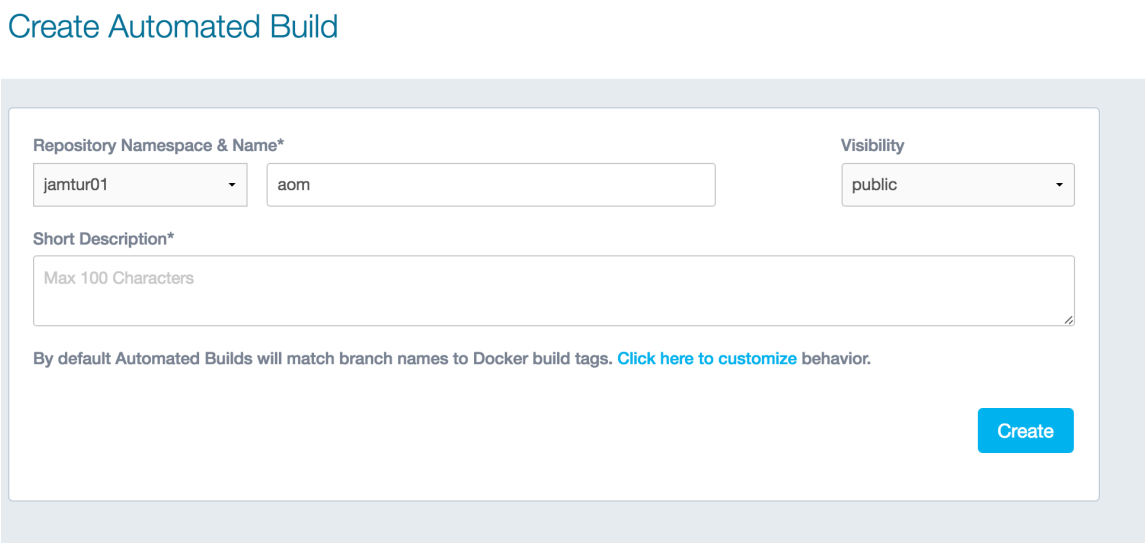

Figure 1.7: Configuring your Automated Build.

Specify the default branch you wish to use, and confirm the repository name.

Specify a tag you wish to apply to any resulting build, then specify the location of the `Dockerfile`. The default is assumed to be the root of the repository, but you can override this with any path.

Finally, click the `Create` button to add your Automated Build to the Docker Hub.

You will now see your Automated Build submitted. Click on the `Build Details` link to see the status of the last build, including log output showing the build process and any errors. A build status of `Done` indicates the Automated Build is up to date. An `Error` status indicates a problem; you can click through to see the log output.

NOTE You can't push to an Automated Build using the `docker push` command. You can only update it by pushing updates to your GitHub or BitBucket repository.